SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Course Code:	CSA6502	**Course Title:**	Generative AI and Large Scale Model
CO Assessed:	CO4-Precision (BL3, BL4, BL5)	**Assessment:**	Assessment Tool 2-Model Comparison Presentation (BERT vs GPT vs T5 vs LLaMA)
Weightage:	10% of CIA	**Total Marks:**	20
CO4 Statement: *Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)*			
Student Name:	C. Praveen kumar	**Reg. No.:**	192472034
Section / Batch:	2024	**Date:**	21-08-2026

Code of Conduct

I, C. Praveen kumar / 192472034 (Name / Reg No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C. Praveen kumar

Name: C. Praveen kumar Reg.No: 192472034

1. Introduction

Transformer-based language models are the foundation of modern natural language processing (NLP), powering applications ranging from search engines and chatbots to machine translation and code generation. Since Vaswani et al. introduced the Transformer architecture in 2017 [1], several influential model families have built on it in different ways. BERT introduced deep bidirectional pretraining for language understanding [2]. GPT popularised decoder-only autoregressive text generation [3][4][5]. T5 unified every NLP task into a single text-to-text framework [6]. LLaMA showed that smaller, openly released decoder-only models trained on more tokens could match or exceed the performance of far larger proprietary models [8][9][10].

This document presents a structured comparison of these four architectures — their design, training objectives, input/output structure, and capabilities — and then evaluates their suitability for multilingual NLP, with particular emphasis on English and Indian languages such as Hindi, Tamil, Telugu, Kannada, Malayalam, Bengali, and Marathi.

2. Objective

- To study and compare the architecture, training objectives, input/output structures, and major capabilities of BERT, GPT, T5, and LLaMA.
- To evaluate the suitability of these four models for multilingual natural language processing, with a specific focus on English and Indian languages.

- To identify the strengths, limitations, and suitable real-world applications of each model, and to provide practical, India-centric recommendations for model selection.

3. Question 1 — Detailed Comparison of BERT, GPT, T5, and LLaMA

3.1 Architecture and Working

3.1.1 BERT — Bidirectional Encoder Representations from Transformers

BERT uses only the encoder half of the Transformer, stacked into 12 layers for BERT-Base (12 attention heads, 768 hidden dimensions, ≈110M parameters) or 24 layers for BERT-Large (16 heads, 1024 dimensions, ≈336M parameters) [2]. Because every encoder layer uses full (non-causal) self-attention, each token's representation is built from context on both its left and right simultaneously — this bidirectionality is what gives BERT its name and its strength at language-understanding tasks. Input tokens are combined with token, segment, and positional embeddings before entering the encoder stack; a special [CLS] token at the start of the sequence is typically used as the aggregate representation for classification tasks.

Conceptual workflow: Input text → WordPiece tokenization → Token + Segment + Position embeddings → Stack of bidirectional Transformer encoder layers → Contextual token embeddings / [CLS] embedding → Task-specific output head (added during fine-tuning).

3.1.2 GPT — Generative Pre-trained Transformer

GPT uses only the decoder half of the Transformer in a causal (left-to-right) configuration [3]. Each self-attention layer is masked so that a token can only attend to itself and the tokens before it, never to future tokens — this is what makes GPT naturally suited to text generation. GPT-1 used a 12-layer decoder (12 heads, 768 dimensions, ≈117M parameters) trained on the BookCorpus dataset [3]; GPT-2 scaled this to 1.5 billion parameters [4]; GPT-3 scaled the same decoder-only design to 175 billion parameters and showed that sufficiently large autoregressive models can perform new tasks from just a natural-language prompt and a handful of examples, without any weight updates ("in-context" or "few-shot" learning) [5].

Conceptual workflow: Input prompt → Byte-Pair-Encoding tokenization → Token + Position embeddings → Stack of causally-masked Transformer decoder layers → Probability distribution over the vocabulary for the next token → Token sampled and appended → process repeats autoregressively until an end-of-sequence token is produced.

3.1.3 T5 — Text-to-Text Transfer Transformer

T5 keeps the full encoder-decoder Transformer design proposed by Vaswani et al. [1], rather than dropping one half as BERT and GPT do [6]. The encoder reads the input bidirectionally (like BERT); the decoder generates the output autoregressively (like GPT) while also attending back to the encoder's output through cross-attention. T5-Base has roughly 220 million parameters — about twice the size of BERT-Base, since it contains two stacks instead of one — and released sizes range from 60 million (Small) to 11 billion (XXL) parameters [6]. T5's defining idea is that every NLP problem — classification, translation, summarization, question answering — is reframed as "input text in, output text out", using short task-prefixes (e.g. "translate English to German:", "summarize:") to tell one single model which task to perform [6][7].

Conceptual workflow: Task-prefixed input text → SentencePiece tokenization → Bidirectional Transformer encoder → Encoder representations → Cross-attention into a causal Transformer decoder → Output generated token-by-token as plain text (label, translation, summary, or answer).

3.1.4 LLaMA — Large Language Model Meta AI

LLaMA is a family of decoder-only causal Transformers, architecturally close to GPT but with three efficiency-focused changes: RMSNorm pre-normalisation instead of LayerNorm, SwiGLU activation functions instead of ReLU, and rotary positional embeddings (RoPE) instead of learned absolute positions [10][11]. LLaMA-1 was released in four dense sizes (7B, 13B, 33B, 65B parameters) trained on up to 1.4 trillion tokens of publicly available text [10]. LLaMA-2 doubled the context length to 4K tokens and added grouped-query attention (GQA) in its largest (70B) model for faster inference [11]. LLaMA-3 applies GQA to all sizes (8B, 70B, 405B), expands the tokenizer vocabulary to 128K tokens, and was trained on about 15.6 trillion tokens, of which roughly 8% is multilingual content [11].

Conceptual workflow: Input prompt (or chat-formatted turns for Instruct variants) → SentencePiece/BPE tokenization with byte-level fallback → Token embeddings + RoPE → Stack of causally-masked decoder layers using RMSNorm and SwiGLU → Next-token distribution → autoregressive generation, same generation loop as GPT.

3.2 Training / Pre-training Objectives

The four models differ most fundamentally in what they are trained to predict. BERT is trained with Masked Language Modeling (MLM) — 15% of input tokens are selected, and of those, 80% are replaced with a [MASK] token, 10% with a random token, and 10% left unchanged; the model must predict the original token — combined with Next Sentence Prediction (NSP), a binary task that judges whether one sentence follows another [2]. GPT is trained purely with causal (autoregressive) language modeling: predict the next token given every token before it [3][4][5]. T5 is trained with a span-corruption denoising objective, where contiguous spans of the input are replaced with a single sentinel token and the decoder must reconstruct the missing spans, phrased in the same text-to-text format used at fine-tuning time [6]. LLaMA, like GPT, is trained with plain causal language modeling over a very large, filtered web-scale corpus, without any auxiliary objective [10][11].

Model	Pre-training Objective	Auxiliary Objective
BERT	Masked Language Modeling (MLM) — predict randomly masked tokens	Next Sentence Prediction (NSP)
GPT	Causal Language Modeling — predict the next token left-to-right	None (GPT-3 adds in-context/few-shot prompting at inference, not training)
T5	Span-corruption denoising in a text-to-text format	Multi-task text-to-text mixture during fine-tuning
LLaMA	Causal Language Modeling — predict the next token left-to-right	None at pre-training; instruction-tuning/RLHF for Chat variants

3.3 Input/Output Structure

Model	Typical Input	Typical Output
BERT	Sentence pair (A, B) with [CLS]/[SEP] markers, or a single sentence	Contextual token embeddings, or a [CLS]-based label/score — not free text generation
GPT	A single prompt sequence of tokens	Next-token probability distribution, sampled autoregressively into free-form generated text
T5	Task-prefixed text string, e.g. "summarize: <article>"	A generated text string — a label, translation, summary, or answer, all expressed as text
LLaMA	A prompt, or structured chat turns (Instruct/Chat variants)	Next-token distribution, sampled autoregressively into generated text, same as GPT

3.4 Major Capabilities

- **BERT:** best suited to language understanding — text classification, named-entity recognition, extractive question answering, natural-language inference, and sentence-similarity tasks. It requires task-specific fine-tuning and is not naturally generative [2].
- **GPT:** strong free-form text generation, conversational ability, in-context/few-shot learning, code generation, and multi-step reasoning, especially from GPT-3 onward [5].
- **T5:** highly versatile — a single fine-tuned model can perform classification, summarization, translation, and question answering, because every task shares the same text-to-text interface [6].
- **LLaMA:** strong general-purpose generation and reasoning at a smaller parameter count than comparable proprietary models, efficient to fine-tune and deploy, and the base for a large open-source ecosystem of derivative chat and instruction-tuned models [10][11].

3.5 Q1 Comparison Table

Feature	BERT	GPT	T5	LLaMA
Developer	Google AI [2]	OpenAI [3][4][5]	Google Research [6]	Meta AI [10][11]
First release	2018	2018 (GPT-1)	2019	2023
Architecture	Encoder-only	Decoder-only (causal)	Encoder–Decoder	Decoder-only (causal)
Attention type	Bidirectional self-attention	Masked (causal) self-attention	Bidirectional (encoder) + causal (decoder)	Masked (causal) self-attention, GQA in larger models
Example sizes	110M (Base) – 336M (Large)	117M (GPT-1) – 175B (GPT-3)	60M (Small) – 11B (XXL)	7B – 405B (across versions)
Pre-training objective	MLM + NSP	Causal LM (next-token)	Span-corruption (text-to-text)	Causal LM (next-token)
Native output	Embeddings / labels	Generated text	Generated text	Generated text
Fine-tuning need	Required per task	Optional (prompting works well)	Required, but one format for all tasks	Optional (prompting or instruction-tuned variants)
Openness	Open weights	Closed (API only, GPT-3 onward)	Open weights	Open weights (community license)

3.6 Real-World Applications and Case Studies

- **BERT:** Google integrated BERT into Search ranking and featured snippets in October 2019, reporting that it improved understanding of natural, conversational, and preposition-sensitive queries for roughly one in ten English-language searches in the U.S. before expanding to more languages [16]. BERT and its derivatives are also widely used in enterprise chatbots, resume/CV screening, and sentiment-analysis pipelines.
- **GPT:** GPT-3.5/GPT-4 power OpenAI's ChatGPT, used for conversational assistance, drafting, coding help, and tutoring at massive scale; the same decoder-only lineage underlies Microsoft Copilot and many API-based customer-support and content-generation products [5].
- **T5:** Google has used T5 and its instruction-tuned derivative FLAN-T5 as research and production baselines for summarization, translation, and multi-task benchmarking, since a single checkpoint can be evaluated across many task types without architectural changes [6].
- **LLaMA:** Because LLaMA's weights are openly released, it has become the base model for a large ecosystem of fine-tuned derivatives — e.g., Alpaca and Vicuna for instruction-following, and Chinese-LLaMA/Alpaca for non-English adaptation — making it popular for academic research and on-premise/offline enterprise deployments where data privacy rules out a hosted API [15].

Question 2 — Multilingual NLP Comparison

1.1 Multilingual Comparison

None of the four base models was designed purely as a multilingual model, but each has multilingual variants or multilingual training data of very different scope. BERT's original release included Multilingual BERT (mBERT), trained on the Wikipedia text of 104 languages using a single shared vocabulary [2]; India-focused variants such as MuRIL improve on mBERT by explicitly augmenting monolingual Indian-language corpora with translated and transliterated pairs, which lets the model handle Latin-script ("Hinglish"-style) transliteration much better than mBERT [17]. GPT models are trained overwhelmingly on English-dominant web text; multilingual ability emerges only as a side effect of incidental non-English text in the crawl, and their tokenizer historically fragmented non-Latin scripts into many more sub-word tokens per word than English [18]. T5's multilingual counterpart, mT5, is deliberately pre-trained on the mC4 corpus covering 101 languages (including Hindi, Tamil, Telugu, Bengali, and other Indian languages), making it one of the more genuinely multilingual options among the four families [7]. LLaMA's pre-training corpus is predominantly English; LLaMA-3 improved this only modestly, with roughly 8% multilingual content in its 15-trillion-token training set [11].

1.2 English vs. Indian-Language Discussion

For English, all four families perform strongly, since English text dominates every pre-training corpus used across BERT, GPT, T5, and LLaMA. For Indian languages, results diverge sharply and depend heavily on tokenizer design and the proportion of Indian-language text seen during pre-training. Because most of these tokenizers were built on English- and Latin-script-heavy corpora, Indian languages — especially morphologically rich, non-Latin-script Dravidian languages such as Tamil, Kannada, Telugu, and Malayalam — are broken into far more sub-word tokens per word than English (a property called low tokenizer "fertility" efficiency). One recent audit reports fertility ratios of about 1.2 tokens/word for English versus 12–16 tokens/word for Tamil, Kannada, Telugu, and Malayalam under the LLaMA-3.1 tokenizer, compared with a much smaller gap (roughly 3–3.5 tokens/word) under the newer GPT-4o tokenizer, which was explicitly re-built with a larger, more multilingual vocabulary [19]. This has three concrete consequences for Indian-language use: (a) the same sentence consumes several times more of the context window, reducing how much text the model can process at once; (b) inference and API costs rise proportionally, since usage is billed per token — one industry analysis found Hindi users paying roughly 60% more than English users for equivalent content under GPT-4o pricing [18]; and (c) because under-tokenized text is also usually under-represented in training, model quality (fluency, factual grounding, hallucination rate) is typically weaker for Indian languages than for English across all four families, though the gap is narrower for models with Indic-specific training such as MuRIL and mT5 than for GPT or LLaMA [17][20].

1.3 Multilingual Comparison Table

Model / Variant	Multilingual Design	Indian-Language Support	Key Limitation
BERT / mBERT / MuRIL	mBERT: 104-language Wikipedia, shared vocabulary [2]. MuRIL: 17 Indian languages + transliteration pairs [17]	MuRIL clearly outperforms mBERT on the XTREME cross-lingual benchmark and on transliterated (Latin-script) Indian-language text [17]	Encoder-only — needs a task-specific head; not a generation model
GPT (GPT-3/3.5/4/4o)	Incidental multilingual coverage from web-scale crawl; newer GPT-4o tokenizer cuts non-English token counts substantially [18][20]	Usable for major Indian languages via prompting, but tokenizer fertility and quality still lag English noticeably [18][19]	Closed model; per-token pricing penalises Indian-language usage
T5 / mT5	mT5 explicitly pre-trained on mC4: 101 languages including Hindi, Tamil, Telugu, Bengali, etc. [7]	One of the strongest "designed-for-multilingual" options among the four; consistent text-to-text interface across languages	Larger multilingual vocabulary increases model/compute size versus English-only T5
LLaMA (1/2/3)	Predominantly English corpus; LLaMA-3 raises multilingual share to only ~8% of 15T tokens [11]	Weakest of the four for Indian languages — very high tokenizer fertility for Tamil/Kannada/Telugu/Malayalam (10–16 tokens/word) [19]	Needs continued pretraining or fine-tuning (e.g. Indic-LLaMA variants) for reliable Indian-language use

1.4 Strengths and Limitations (Multilingual Context)

BERT / mBERT / MuRIL

- Strength: strong at understanding tasks (classification, NER, extractive QA) in Indian languages when using MuRIL, which is purpose-built for 17 Indian languages and transliterated text [17].
- Limitation: encoder-only design cannot natively generate fluent multilingual text (translation, summarisation); mBERT alone under-represents low-resource Indian languages relative to MuRIL [17].

GPT

- Strength: fluent generative and conversational ability in major Indian languages via prompting, with no fine-tuning needed; newer tokenizers have narrowed (not closed) the non-English cost/efficiency gap [20].
- Limitation: closed-source, usage-based pricing that is structurally more expensive for Indian-language text than English; quality still trails English on nuanced or culturally specific content [18].

T5 / mT5

- Strength: purpose-built multilingual coverage (101 languages) with a single consistent text-to-text interface, useful for translation and summarisation involving Indian languages [7].
- Limitation: generation quality and fluency for a specific Indian language can lag a well-tuned monolingual or Indic-specific model, since capacity is shared across all 101 languages.

LLaMA

- Strength: open weights make it possible to continue pre-training or fine-tune on Indian-language corpora at low cost, and to deploy fully offline for data-sovereignty-sensitive applications.
- Limitation: out of the box it is the weakest of the four for Indian languages, given its low (~8%) multilingual training share and very high tokenizer fertility for Dravidian languages in particular [11][19].

1.5 Suitable Applications

Model	Best-fit English Applications	Best-fit Indian-Language Applications
BERT / MuRIL	Search relevance, document classification, extractive QA	Regional-language chatbots for understanding-only tasks; MuRIL suits Indic sentiment analysis and NER
GPT	Conversational assistants, drafting, coding help, summarisation	Multilingual customer support and content generation, where API cost for Indian-language traffic is acceptable
T5 / mT5	Unified summarisation/translation/classification pipelines	English↔Indian-language machine translation and cross-lingual summarisation
LLaMA	On-premise assistants, research, fine-tuned domain chatbots	Base for custom-trained Indic LLMs where the corpus is fine-tuned/continued-pretrained on target-language data

1.6 Final Recommendations

- For Indian-language understanding tasks (classification, NER, extractive QA) with limited compute, MuRIL is the most India-appropriate choice among the four families, given its purpose-built training on 17 Indian languages and transliterated text [17].
- For Indian-language generation and translation (summarisation, machine translation), mT5 is preferable to GPT or LLaMA because it was explicitly trained on mC4 across 101 languages with a consistent text-to-text interface [7].
- For conversational, general-purpose assistants where development speed matters more than per-token cost, GPT remains a strong default, provided the higher cost of Indian-language tokens is budgeted for [18][20].

- For privacy-sensitive, offline, or heavily customised deployments, LLaMA is the best starting point, but it should be continued-pretrained or fine-tuned on Indian-language data before production use, since its out-of-the-box multilingual coverage is the weakest of the four [11][19].

2. Conclusion

BERT, GPT, T5, and LLaMA all descend from the same Transformer architecture [1], but their design choices — encoder-only, decoder-only, or encoder-decoder — make them suited to different tasks: BERT for language understanding, GPT and LLaMA for open-ended generation, and T5 for a single unified interface across many tasks. For multilingual and Indian-language applications specifically, no single model is uniformly best: MuRIL and mT5 were deliberately built with Indic-language coverage and outperform the general-purpose GPT and LLaMA models on Indian-language benchmarks, while GPT and LLaMA offer broader ecosystem support and generative flexibility at the cost of higher tokenization overhead and weaker out-of-the-box performance for Indian languages. The right choice therefore depends on the task (understanding vs. generation), the target languages, deployment constraints (hosted API vs. on-premise), and budget — with purpose-built Indic models being the safer default wherever Indian-language quality is the primary requirement.

3. References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems (NeurIPS), 30.

[2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019.

[3] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI.

[4] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI.

[5] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems (NeurIPS), 33.

[6] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research, 21(140), 1–67.

[7] Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. Proceedings of NAACL-HLT 2021.

[8] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. Meta AI.

[9] Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. Meta AI.

[10] AI @ Meta, Llama Team (2024). The Llama 3 herd of models. Meta AI.

[11] Khanuja, S., Bansal, D., Mehtani, S., et al. (2021). MuRIL: Multilingual representations for Indian languages. arXiv:2103.10730.

[12] Kakwani, D., Kunchukuttan, A., Golla, S., N.C., G., Bhattacharyya, A., Khapra, M. M., & Kumar, P. (2020). IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages. Findings of EMNLP 2020.

[13] Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). Unsupervised cross-lingual representation learning at scale. Proceedings of ACL 2020.

[14] Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). Language model tokenizers introduce unfairness between languages. arXiv:2305.15425.

[15] Rozière, B., et al. (2023). Code Llama: Open foundation models for code. Meta AI. (Representative of the open-source LLaMA derivative ecosystem, alongside Alpaca and Vicuna.)

[16] Nayak, P. (2019, October 25). Understanding searches better than ever before. The Keyword, Google Blog.

[17] Khanuja, S., et al. (2021). MuRIL: Multilingual representations for Indian languages [XTREME benchmark results]. arXiv:2103.10730.

[18] Kathane, R. (2025). Not speaking English to ChatGPT costs you millions of tokens. (Industry analysis of GPT-4o per-language token pricing.)

[19] VerChol (2026). Grammar-first tokenization for agglutinative languages [tokenizer fertility benchmark for Hindi, Tamil, Kannada, Telugu, Malayalam across LLaMA-3.1, GPT-4o, and Gemma-2]. arXiv:2603.05883.

[20] Cultural Fidelity in Large-Language Models (2024). Evaluation of GPT-4o tokenizer efficiency gains for Hindi and other non-English languages. arXiv:2410.10489.